CS2810 : Lab 6 Spell Checker

1 Problem Statement

Given a `wordlist`, implement a spell checker that converts a given `query` word into a correct word based on a **specific hierarchy** of matching rules. For a list of queries, return a corresponding list of corrected words.

2 Matching Rules

The spell checker evaluates each query against the `wordlist` according to the following four operations, **strictly in the below order of priority**. If multiple matches exist, return the **first** one that appears in the `wordlist`.

1. **Exact Match:** If the query exactly matches a word in the wordlist (**case-sensitive**), print that word.
   Example: `wordlist = ["KiTe", "kite"]`, `query = "kite"` → `"kite"`.
2. **Capitalization Error:** If the query matches a word in the wordlist **case-insensitively**, print the first such match.
   Example: `wordlist = ["KiTe", "kite"]`, `query = "KITE"` → `"KiTe"`.
3. **Anagram Match:** If the query matches a word (**case-insensitive**) by rearranging characters, print the first such match.
   Example: `wordlist = ["listen", "silent"]`, `query = "enlist"` → `"listen"`.
4. **Vowel Error:** If the query matches a word (**case-insensitive**) after replacing vowels (`a, e, i, o, u`) with any other vowel, print the first such match.
   Example: `wordlist = ["yellow"]`, `query = "yollow"` → `"yellow"`.

If no match is found after all four operations, print an empty string `""`.

3 Input Format

The input is provided in the following format:

- A single line containing two integers n and m, representing the size of the `wordlist` and `queries` respectively.
- A single line containing n space-separated strings representing the `wordlist`.
- A single line containing m space-separated strings representing the `queries`.

4 Constraints & Weightage

- $1 \leq n, m \leq 10,000$.
- $1 \leq \text{wordlist}[i].\text{length}, \text{queries}[i].\text{length} \leq 7$.
- All strings consist only of English letters (uppercase and lowercase) and digits from 0-9.

Operation Type	Weightage
Exact Match	30%
Capitalization Error	30%
Anagram Match	20%
Vowel Error	20%

5 Sample Tests

Sample 1:

Input:

```
4 3
KiTe kite hare Hare
kite KITE keti
```

Output:

```
kite KiTe KiTe
```

Explanation: "kite" is an exact match (Rule 1). "KITE" matches "KiTe" via capitalization (Rule 2). "keti" contains the same characters as "KiTe" and thus matches via anagram logic (Rule 3).

Sample 2:

Input:

```
2 2
apple pale
lepa pleap
```

Output:

```
pale apple
```

Explanation: "lepa" has the same characters as "pale". "pleap" has the same characters as "apple" (one 'a', two 'p's, one 'l', one 'e'). It matches via Rule 3.

Sample 3:

Input:

```
2 1
Tea ate
ate
```

Output:

```
ate
```

Explanation: Although "ate" is an anagram of "Tea", the exact match "ate" has higher priority (Rule 1).